

Annealed Kernel-Tabular Q-learning: Tabular Convergence with Kernel-based Generalization

1 Project Summary

The goal of this project is to design a reinforcement learning algorithm that propagates TD errors to unvisited, similar states using an exponential kernel based on a distance metric in a finite metric space, while maintaining the convergence properties of tabular Q-learning.

The core idea is to maintain a global tabular Q-table Q^G , introduce an auxiliary kernel estimator Q^K , and use the following combined Q-function for actual action selection and TD targets:

$$\tilde{Q}_t(s, a) = Q_t^G(s, a) + \beta_t(s, a)Q_t^K(s, a) \quad (1)$$

Here, $\beta_t(s, a)$ is annealed such that $\beta_t(s, a) \rightarrow 0$. Therefore, in the early stages of learning, Q^K provides a generalization bias, and in the later stages, its influence vanishes, recovering the tabular Q-learning convergence of Q^G .

2 Problem Setting

Consider a finite MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$. We assume $|\mathcal{S}| < \infty$, $|\mathcal{A}| < \infty$, $\gamma \in [0, 1)$, and the rewards are bounded such that $|r_t| \leq R_{max}$.

Additionally, we assume that the state space $\mathcal{S}$ is equipped with an arbitrary distance metric $d(x, y)$.

3 Exponential Kernel

The exponential kernel capturing the similarity between states is defined as follows:

$$k(x, y) = \exp(-\lambda d(x, y)) \quad (2)$$

The search neighborhood for the current state s_t is uniformly defined using a radius R :

$$\mathcal{N}(s_t) = \{x \in \mathcal{S} : d(s_t, x) \leq R\} \quad (3)$$

4 Auxiliary Kernel Estimator

The auxiliary kernel table Q^K is maintained with a single table size of $|\mathcal{S}| \times |\mathcal{A}|$ over the global state space. The combined Q-function used for action selection and TD target computation consists of the combination of the global table and the kernel table.

5 Combined Q Function

We define the value used for action selection and TD targets by combining the global tabular Q-table Q_t^G and the auxiliary estimator Q_t^K :

$$\tilde{Q}_t(s, a) = Q_t^G(s, a) + \beta_t(s, a)Q_t^K(s, a) \quad (4)$$

The visitation count-based annealing schedule is given by $\beta_t(s, a) = \frac{\beta_0}{(1+N_t(s, a))^p}$. Since $N_t(s, a) \rightarrow \infty$ as all (s, a) are visited infinitely often, we have $\beta_t(s, a) \rightarrow 0$.

6 Algorithm

6.1 TD Error with Kernel Term

After observing the transition (s_t, a_t, r_t, s_{t+1}) , the TD error is calculated as:

$$\delta_t = r_t + \gamma \max_{a'} \tilde{Q}_t(s_{t+1}, a') - \tilde{Q}_t(s_t, a_t) \quad (5)$$

6.2 Global Q-table Update

The global table is directly updated using the TD error, while all other entries remain unchanged:

$$Q_{t+1}^G(s_t, a_t) = Q_t^G(s_t, a_t) + \alpha_t(s_t, a_t)\delta_t \quad (6)$$

6.3 Flat Kernel Update

The auxiliary kernel table Q^K performs a broadcasting update to the entire neighborhood state set using kernel weights. By default, the kernel function $k(s_t, x)$ can be used directly:

$$Q_{t+1}^K(x, a_t) = Q_t^K(x, a_t) + \alpha_t^K k(s_t, x)\delta_t \quad \text{for all } x \in \mathcal{N}(s_t) \quad (7)$$

Optionally, to prevent the update from diverging, a locally normalized kernel weight $\bar{k}$, where the sum within the neighborhood equals 1, can be applied instead:

$$\bar{k}(s_t, x) = \frac{\exp(-\lambda d(s_t, x))}{\sum_{z \in \mathcal{N}(s_t)} \exp(-\lambda d(s_t, z))} \quad (8)$$

In this case, $k(s_t, x)$ in the update equation is replaced by $\bar{k}(s_t, x)$.

7 Pseudocode

Algorithm 1 Non-Factored Annealed Kernel-Tabular Q-learning

```

1: Initialize  $Q^G(s, a) = 0$ ,  $Q^K(s, a) = 0$ ,  $N(s, a) = 0$  for all  $s \in \mathcal{S}, a \in \mathcal{A}$ 
2: for each episode do
3:   Observe  $s_t$ 
4:   while  $s_t$  is not terminal do
5:     for all  $a \in \mathcal{A}$  do
6:        $\tilde{Q}(s_t, a) = Q^G(s_t, a) + \beta_t(s_t, a)Q^K(s_t, a)$ 
7:     end for
8:     Choose  $a_t$  by  $\epsilon_t$ -greedy w.r.t.  $\tilde{Q}(s_t, \cdot)$ 
9:     Execute  $a_t$ , observe  $r_t, s_{t+1}$ 
10:     $N(s_t, a_t) \leftarrow N(s_t, a_t) + 1$ 
11:     $\delta_t = r_t + \gamma \max_{a'} \tilde{Q}(s_{t+1}, a') - \tilde{Q}(s_t, a_t)$ 
12:    % Global Update
13:     $Q^G(s_t, a_t) \leftarrow Q^G(s_t, a_t) + \alpha_t(s_t, a_t)\delta_t$ 
14:    % Flat Kernel Update (optional: use  $\bar{k}$  instead of  $k$ )
15:    for each  $x \in \mathcal{N}(s_t)$  do
16:       $Q^K(x, a_t) \leftarrow Q^K(x, a_t) + \alpha_t^K k(s_t, x)\delta_t$ 
17:    end for
18:     $s_t \leftarrow s_{t+1}$ 
19:  end while
20: end for

```

8 Convergence Analysis

The convergence proof of this algorithm proceeds by reducing the update equation of the global value function Q_t^G to an Asynchronous Stochastic Approximation form, where a vanishing perturbation is added to the pure Tabular Q-learning operation.

8.1 Assumptions

To guarantee convergence, we reuse the convergence conditions of traditional Tabular Q-learning:

Assumption 1. 1. The state space $\mathcal{S}$ and action space $\mathcal{A}$ are finite ($|\mathcal{S}| < \infty, |\mathcal{A}| < \infty$).

2. Rewards are bounded ($|r_t| \leq R_{max}$).

3. All state-action pairs (s, a) are visited infinitely often ($\lim_{t \rightarrow \infty} N_t(s, a) = \infty$).

4. The learning rate α_t satisfies the Robbins-Monro conditions ($\sum_t \alpha_t(s, a) = \infty, \sum_t \alpha_t^2(s, a) < \infty$).

5. The auxiliary kernel table is bounded, and the annealing coefficient vanishes ($\|Q_t^K\|_\infty \leq B_K < \infty, \lim_{t \rightarrow \infty} \beta_t(s, a) = 0$).

8.2 TD Error Decomposition and Perturbation Bounding

To compare with the standard Tabular Q-learning update operator, we define a hypothetical TD error δ_t^G using only the pure Q_t^G as follows:

$$\delta_t^G = r_t + \gamma \max_{a'} Q_t^G(s_{t+1}, a') - Q_t^G(s_t, a_t) \quad (9)$$

The actual δ_t used in the algorithm can be decomposed into the sum of δ_t^G and a perturbation term e_t as follows:

$$\delta_t = \delta_t^G + e_t \quad (10)$$

The algebraic definition of e_t is given by:

$$e_t = \gamma \left[\max_{a'} \tilde{Q}_t(s_{t+1}, a') - \max_{a'} Q_t^G(s_{t+1}, a') \right] - \beta_t(s_t, a_t) Q_t^K(s_t, a_t) \quad (11)$$

Applying the triangle inequality, the upper bound of the absolute value of e_t is obtained as follows:

$$|e_t| \leq \gamma \left| \max_{a'} \tilde{Q}_t(s_{t+1}, a') - \max_{a'} Q_t^G(s_{t+1}, a') \right| + |\beta_t(s_t, a_t) Q_t^K(s_t, a_t)| \quad (12)$$

Here, we apply the property of the supremum norm (sup-norm), where $|\max f - \max g| \leq \max |f - g|$ holds for any two functions f and g , to the first term:

$$\begin{aligned} \left| \max_{a'} \tilde{Q}_t(s_{t+1}, a') - \max_{a'} Q_t^G(s_{t+1}, a') \right| &\leq \max_{a'} |\tilde{Q}_t(s_{t+1}, a') - Q_t^G(s_{t+1}, a')| \\ &= \max_{a'} |\beta_t(s_{t+1}, a') Q_t^K(s_{t+1}, a')| \\ &\leq \|\beta_t Q_t^K\|_\infty \end{aligned} \quad (13)$$

Substituting this, the upper bound of the perturbation term e_t is bounded via the maximum norm in the global space as follows:

$$|e_t| \leq \gamma \|\beta_t Q_t^K\|_\infty + \|\beta_t Q_t^K\|_\infty = (1 + \gamma) \|\beta_t Q_t^K\|_\infty \quad (14)$$

By assumption, $\|Q_t^K\|_\infty \leq B_K$ and $\beta_t \rightarrow 0$; therefore, the following property is strictly derived:

$$\lim_{t \rightarrow \infty} \|\beta_t Q_t^K\|_\infty = 0 \implies \lim_{t \rightarrow \infty} e_t = 0 \quad \text{almost surely (a.s.)} \quad (15)$$

8.3 Contraction Mapping and Stochastic Approximation

The Bellman optimality operator $T : \mathbb{R}^{|S| \times |A|} \rightarrow \mathbb{R}^{|S| \times |A|}$ is defined as follows:

$$(TQ)(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q(s', a') \mid s, a] \quad (16)$$

Since $\gamma < 1$, the operator T is a contraction mapping with a unique fixed point Q^* under the maximum norm.

Now, we reconstruct the update equation of the global table Q^G using the operator T , a martingale difference noise w_t , and the perturbation term e_t :

$$Q_{t+1}^G(s_t, a_t) = Q_t^G(s_t, a_t) + \alpha_t(s_t, a_t) [(TQ_t^G)(s_t, a_t) - Q_t^G(s_t, a_t) + w_t(s_t, a_t) + e_t] \quad (17)$$

Here, $w_t(s_t, a_t) = r_t + \gamma \max_{a'} Q_t^G(s_{t+1}, a') - (TQ_t^G)(s_t, a_t)$, which possesses a martingale structure where its conditional expectation with respect to the past history is 0 ($\mathbb{E}[w_t \mid \mathcal{F}_t] = 0$).

Theorem 1. *According to the Stochastic Approximation Theorem, since T is a contraction mapping and e_t converges to 0 almost surely, the vanishing perturbation term does not affect the asymptotic dynamics of the system. Therefore, the following holds almost surely (a.s.):*

$$\lim_{t \rightarrow \infty} Q_t^G = Q^* \quad (18)$$

Furthermore, the convergence of the actual agent's combined value function $\tilde{Q}_t$ is also guaranteed:

$$\lim_{t \rightarrow \infty} \tilde{Q}_t = \lim_{t \rightarrow \infty} (Q_t^G + \beta_t Q_t^K) = \lim_{t \rightarrow \infty} Q_t^G + \lim_{t \rightarrow \infty} \beta_t Q_t^K = Q^* + 0 = Q^* \quad (19)$$

9 Complexity Analysis

Time Complexity: Action evaluation and TD target computation take $O(A)$ in a single step. The kernel auxiliary update takes $O(|\mathcal{N}(s_t)|)$, which corresponds to the neighborhood size. Thus, the total 1-step time complexity is as follows:

$$T_{step} = O(A + |\mathcal{N}(s_t)|) \quad (20)$$

Memory Complexity: Due to the global tables Q^G and Q^K , the overall memory is dominated by $O(A|S|)$.

10 Derivation of Kernel Broadcast via RKHS

The broadcast update rule of the auxiliary kernel estimator Q^K is mathematically naturally derived via the functional gradient in a Reproducing Kernel Hilbert Space (RKHS).

1. Q-function in RKHS

Assume that the kernel estimator $Q^K(\cdot, a)$ for an arbitrary action a belongs to an RKHS $\mathcal{H}$ defined by a positive definite kernel $k(x, y)$. By the reproducing property of RKHS, the value function at state s_t is expressed as an inner product with the kernel function:

$$Q^K(s_t, a) = \langle Q^K(\cdot, a), k(s_t, \cdot) \rangle_{\mathcal{H}} \quad (21)$$

2. Objective Function and Semi-gradient

Let L be the squared TD error loss function for the transition observed at a single time step t :

$$L(Q^K) = \frac{1}{2} \delta_t^2 = \frac{1}{2} \left(r_t + \gamma \max_{a'} \tilde{Q}(s_{t+1}, a') - \tilde{Q}(s_t, a_t) \right)^2 \quad (22)$$

Applying a semi-gradient approach that treats the TD target as a constant, similar to standard Q-learning, the functional derivative of the loss function with respect to the function $Q^K(\cdot, a_t)$ is calculated by the chain rule and the reproducing property as follows:

$$\nabla_{Q^K} L = \frac{\partial L}{\partial Q^K(s_t, a_t)} \nabla_{Q^K} Q^K(s_t, a_t) = -\delta_t k(s_t, \cdot) \quad (23)$$

3. Functional Gradient Descent

Updating the function itself (gradient descent) in the direction that minimizes the loss in the RKHS yields the following:

$$Q_{t+1}^K(\cdot, a_t) = Q_t^K(\cdot, a_t) - \alpha_t^K \nabla_{Q^K} L = Q_t^K(\cdot, a_t) + \alpha_t^K \delta_t k(s_t, \cdot) \quad (24)$$

4. Pointwise Evaluation (Spatial Broadcasting)

Evaluating the updated new function $Q_{t+1}^K(\cdot, a_t)$ at an arbitrary unvisited state x yields the following pointwise update rule:

$$Q_{t+1}^K(x, a_t) = Q_t^K(x, a_t) + \alpha_t^K \delta_t k(s_t, x) \quad (25)$$

In conclusion, the functional derivative in RKHS mathematically proves that δ_t must be spatially broadcasted proportionally to $k(s_t, x)$, which is the geometric similarity between the two states. In actual algorithm implementation, as discussed in the previous section, the selectively locally normalized $\bar{k}(s_t, x)$ can be used instead for update stability and mass conservation.